

Chapter 3

File Systems and Management of Data Storages

In a computer, a file system is the way in which files are named and where they are placed logically for storage and retrieval. Without a file system, stored information wouldn't be isolated into individual files and would be difficult to identify and retrieve. As data capacities increase, the organization and accessibility of individual files are becoming even more important in data storage.

Digital file systems and files are named for and modeled after paper-based filing systems using the same logic-based method of storing and retrieving documents.

File systems can differ between operating systems (OS), such as Microsoft Windows, macOS and Linux-based systems. Some file systems are designed for specific applications. Major types of file systems include distributed file systems, disk-based file systems and special purpose file systems.

3.1 File system administration

A file system stores and organizes data and can be thought of as a type of index for all the data contained in a storage device. These devices can include hard drives, optical drives and flash drives.

File systems specify conventions for naming files, including the maximum number of characters in a name, which characters can be used and, in some systems, how long the file name suffix can be. In many file systems, file names are not case sensitive.

Along with the file itself, file systems contain information such as the size of the file, as well as its attributes, location and hierarchy in the directory in the metadata. *Metadata* can also identify free blocks of available storage on the drive and how much space is available.

A file system also includes a format to specify the path to a file through the structure of directories. A file is placed in a directory -- or a folder in Windows OS -- or subdirectory at the desired place in the tree structure. PC and mobile OSes have file systems in which files are placed somewhere in a hierarchical tree structure.

3.1.1 Partitioning disks with fdisk and parted command

Creating disk partitions enables you to **split your hard drive** into multiple sections that act independently.

In Linux, users **must structure** storage devices (USB and hard drives) before using them. Partitioning is also useful when you are installing multiple operating systems on a single machine.

Before files and directories are created on the storage medium, **partitions** should be put into place. A **partition** is a region of the hard disk or other storage that the OS manages separately. One file system is contained in the primary partition, and some OSes allow for multiple partitions on one disk. In this situation, if one file system gets **corrupted**, the data in a different partition **will be safe**.

The **fdisk** and **parted** commands are used to manipulate storage devices. The **fdisk** and **parted** commands perform the same function, but **parted** has more options.

The fdisk Command

***fdisk* is a menu-guided program for creating and manipulating partition tables. It understands MS-DOS-type partition tables and other disk labels. Initially, *fdisk* wasn't designed for large partitions** because it couldn't accept GUID (globally unique identifiers) partition tables. But, the recent version supports GPT (GUID partition table) though using MBR (Master Boot Record) (for drives less than 2TB) is recommended.

GPT support is currently new and, therefore, it's deemed an experimental phase. Further, the latest versions of *fdisk* criticized the use of cylinders as the default display unit and MS-DOS compatibility by default. *fdisk* has an interactive command-line menu that allows us to use letters to specify actions:

```
$ sudo fdisk /dev/sda
```

When using *fdisk*, the sectors must be specified in absolute terms using sector numbers. If we're creating a sector, the size must be set in absolute terms to

specify the beginning of that sector. For instance, using 80M as the first sector of a disk indicates that the sector is 80 MiB from the beginning of the disk.

When we're using the absolute position in sector numbers or as positions, the measuring unit (its size) should be in kibibytes (K), mebibytes (M), gibibytes (G), and tebibytes (T). Also, it's possible to express relative terms by preceding the size with either a +size or a -size. For example, a +1G indicates a point 1 GiB after the start sector, while a -100M indicates a point 100 MiB before the last available sector.

fdisk performs partition alignment automatically on the one-megabyte boundary if DOS compatibility mode has been disabled, making it compatible with various Advanced Format HDDs (hard disk drives) and SSDs (Solid-state drives). The default settings should give us accurate alignment.

The *parted* Command

parted is an **alternative command** to *fdisk*. It's a utility we use to manipulate disk partitions and **helps us to create space for new operating systems, reorganize disk usage, and copy data to new hard disks.** *parted* comes with **better functionality** than *fdisk*, such as resizing (shrinking and extending partitions). In addition, it supports multiple partition table formats, including MS-DOS and GPT.

We can install it by installing the *parted* package. **It has two modes: command-line and interactive. The interactive mode makes partitioning simple as it reduces unnecessary repetition by automatically applying all partitioning commands to the selected device.** If we're using the interactive mode, the prompt changes from a dollar sign (\$) to (parted):

```
$ sudo parted /dev/sda
```

On the other hand, if we're using the command-line mode, we run the *parted* command together with the right options. If the command is successful, it doesn't throw an error and the cursor moves to a new line. We must specify the partition/drive we want to run the command on: the syntax for

creating a new partition:

```
$ sudo parted mkpart part-type name fs-type start end
```

Let's have a look at the significant differences between these commands:

<i>Fdisk</i>	<i>parted</i>
When we run <i>fdisk</i> , the output is displayed using the partition type.	Uses the file system to display partitions available in a storage device.
Performs partition alignment automatically on the one-megabyte boundary, but MS-DOS compatibility mode has to be disabled.	When creating a partition, might warn about improper partition alignment but doesn't alert when the alignment is proper, and it only aligns the partition start, not the size.
Doesn't give us the option to shrink or extend the partition.	We can extend a partition to utilize the unused disk space or shrink a partition to use its capacity for other purposes.
Only has an interactive mode in which we use letters to specify actions.	Has both interactive and command-line modes. In both of these modes, we have to specify the right commands with the required options.

We've looked at both *fdisk* and *parted* commands. We've mentioned the modes each of these commands has and the kind of file systems they can support. The interactive mode should be the most preferred when using *parted* as it simplifies the process of creating partitions.

fdisk supports automatic alignment only if DOS is turned off while *parted* only aligns the partition start. *fdisk* is present in most distros while *parted* is missing in some distributions and must be installed to be used. Lastly, we should be mindful when using either *fdisk* or *parted* as misuse can result in the loss of important data.

3.1.2 Creating, mounting and maintaining file systems

Creating a file system **writes information** to the device and creates order of the empty space. This file system–related data consumes a small percentage of the space. The remaining space on the disk drive is split into small, consistently sized segments called **blocks**. Linux supports a number of file system types, some of which are described as follows.

File system	Description
ext2	High performance for fixed disk and removable media
ext3	Journaling version of ext2
ext4	Supports larger files and file system sizes
vfat	MS-DOS file system useful when sharing files between Windows and Linux
XFS	High-performance journaling file system
Btrfs	Addresses scalability requirements of large storage systems

Creating Filesystems

The command to build a Linux file system on a device, or hard disk partition, is ***mkfs***. The syntax for the command is:

```
# mkfs [options] device
```

The **mkfs** command is actually a front end for the different file system builder utilities such as **mkfs.ext2** and **mkfs.ext4**. These utilities are executable directly from the command line. When using the **mkfs** wrapper, include the **-t fs type** option to specify the type of file system to be built. If not specified, the default file system type, **ext2**, is created.

Mounting File Systems

File systems on different partitions and removable devices, such as CDs, DVDs, or USB flash drives, **must be attached** to the directory hierarchy to be accessed. To attach a partition or device, a **mount point** must be created. A mount point is simply a directory created with the `mkdir` command. After a directory, or mount point, is created, attach the partition by using the **mount command**. Syntax for the mount command is:

```
# mount [options] device_file mount_point
```

The following example creates a mount point (`/test`) and attaches the partition:

```
# mkdir /test
```

```
# mount /dev/xvdf1 /test
```

To specify mount options, use the `-o` flag followed by a comma-separated string of options. The following are some of the available options for the mount command:

- `auto`: Allows the file system to be mounted automatically by using the `mount -a` command
- `loop`: Mounts the image as a loop device
- `noauto`: Disallows the automatic mount of the file system by using the `mount -a` command
- `noexec`: Disallows the execution of binary files on the file system
- `nouser`: Disallows an ordinary user (other than root) to mount and unmount the file system
- `remount`: Remounts the file system in case it is already mounted
- `ro`: Mounts the file system for reading only
- `rw`: Mounts the file system for both reading and writing
- `user`: Allows an ordinary user (other than root) to mount and unmount the file system

For example, to mount the `/dev/xvdf1` partition on the `/test` mount point as read-only with only the root user able to mount and unmount the file system, enter:

```
# mount -o nouser,ro /dev/xvdf1 /test
```

Unmounting File Systems

To unmount a file system, use the **umount** command. The partition name, the device name, or the mount point is used as an argument.

Example: `# umount /dev/xvdd1`

```
# umount /test
```

3.1.3 Swap

Linux divides its **physical RAM** (random access memory) into chunks of memory called **pages**. **Swapping** is the process whereby a page of memory is copied to the **preconfigured space** on the **hard disk**, called **swap space**, to free up that page of memory. The combined size of the **physical memory** and the **swap space** is the amount of **virtual memory** available.

What Is Swap Space?

Swap space is space on a **hard drive** (HDD or SSD) that represents a substitute for **physical (RAM) memory**. This feature allows an operating system to temporarily move **inactive** or **less frequently** used memory pages from RAM to a designated area on the hard drive and it allows processes to continue running when RAM is fully used and prevents memory errors.

Swap frees up RAM for more important tasks that require more processing power by transferring data to and from a designated disk space. The data interchange is called **swapping**, while the designated space is called **swap space**.

Swap space also enables **hibernation** and **safeguards** critical processes by temporarily offloading data. However, it should only be a complement to RAM because a system that relies on swap would suffer significant performance degradation.

Operating systems like **Windows** or **Linux** provide a certain amount of swap space by default, which users can later change in accordance with their requirements. Users can also disable swap space, but that means the kernel must kill some processes to create enough free RAM for new processes.

Types of Swap Space

There are two types of swap space in Linux:

1. Traditional Swap Space

The **classic form** of swap space that has been in use for decades is **traditional swap space**. It involves the designation and use of a **dedicated partition** on a hard drive. A swap partition is **formatted** specifically for this purpose and is separate from the main system partitions.

Traditional swap space is suitable for scenarios where you have a dedicated server with specific disk partitions and need a **fixed amount** of swap space. Using swap space is **common in server environments** where a portion of the storage device is allocated for swap and isolated from the rest of the file system.

Traditional swap spaces are also useful in **performance-critical** systems because they offer better performance than a swap file. Since swap space is fixed, it also provides predictable behavior, which is an advantage when you must ensure a specific amount of swap is always available.

2. Swap File

A swap file is a file on the file system that the OS uses as swap space. Swap files offer **more flexibility** because users can **create, resize, or remove** the file without having to perform disk repartitioning.

In Linux, there are two types of swap files:

- **Temporary swap file**. It typically uses **fragmented disk space** and **doesn't reserve** a part of the hard drive, making it suitable for limited disk space.

- **Permanent swap file.** It takes up a contiguous section of the hard drive, meaning it needs more disk space than a temporary swap file. The advantage of using a permanent swap file is that it requires fewer I/O operations, which makes it less resource-intensive than a temporary swap file.

Swap files are often easier to manage than traditional swap spaces, and they can even be placed on different storage devices, which provide greater control over swap space management. Their flexibility makes them great for scenarios with limited disk space.

Lastly, swap files are particularly advantageous for virtualized environments. Virtual machines often have dynamic memory requirements, and swap files allow users to easily adjust swap space without altering the underlying disk setup.

Benefits of Using Swap Space

Using swap space provides multiple advantages. Some of them are:

- Preventing Memory-Related Crashes.
- Memory Over commitment.
- Large Programs and Multitasking Support.
- Improving Stability.
- Flexibility.
- Virtual Memory Management.
- Resource Allocation.

Although swap space offers many advantages, relying heavily on it leads to degraded performance, as accessing data stored on a hard drive is slower compared to RAM. Therefore, balancing swap space with an appropriate amount of physical RAM is the best way to ensure optimal system performance.

3.1.4 Determining disk usage with df and du

Determining disk usage is crucial for managing storage effectively and

preventing disk space from **running out**. Two essential tools for this task are `df` and `du`.

df Command

The `df` command stands for “**disk free**” and displays information about the file system’s **disk space usage**.

The `df` command provides **an overview** of disk usage for each **mounted** file system. It displays information such as:

- File system: The name of the mounted filesystem
- Total: The total disk space available on the filesystem
- Used: The amount of disk space currently used on the filesystem
- Available: The amount of disk space that is still available
- Use%: The percentage of disk space that is being used

The `df` command has **several options** that can be used to **customize** the output.

Some of the most commonly used options include:

- `-h`: displays the output in a human-readable format
- `-T`: displays the file system type
- `-i`: displays the inode information

du Command

The `du` command stands for “**disk usage**” and displays the disk space a file or directory uses. The command displays the **total amount** of disk space used by the specified **file or directory** and its subdirectories.

The `du` command has several options that can be used to customize the output. Some of the most commonly used options include:

- `-h`: displays the output in a human-readable format
- `-s`: displays only the total disk space used

❖ `df` (disk free) and `du` (disk usage) are both commands used to **check disk usage** on Linux systems. However, they **differ** in their scope and purpose.

`df` provides an overview of disk usage for each mounted file system. It displays the total, used, available, and percentage of used space for

each file system.

The output of `df` looks something like this:

Filesystem	Total	Used	Free	%
/dev/sda1	100G	80G	20G	80%
/dev/sda2	50G	30G	20G	60%

du provides a more **detailed breakdown** of disk usage for a specific directory or file. It displays the total size of the specified directory or file, along with the size of each file and subdirectory within it.

The output of `du` looks something like this:

```
/home/user
100M total
 20M .bash_history
 10M .bashrc
  5M .vimrc
 75M Documents
 50M documents
 25M images
```

3.1.5 Configuring disk quotas

Disk quotas allow system administrators to **control and limit** the amount of disk space that **users or groups** can utilize on a file system. By configuring disk quotas, you can prevent users from **consuming excessive disk space** and ensure fair resource allocation.

3.2 Logical volume management (LVM) and RAID

Logical Volume Management (LVM) is a method of managing disk storage that provides a **flexible** and **extensible** way to store and manage data. LVM allows you to create **logical volumes**, which are **virtual partitions** that can span multiple physical disks. This means that you can use LVM to **pool together multiple disks** to create a single, large logical volume.

LVM provides several advantages over traditional partitioning, including:

- **Flexibility**: LVM allows you to easily resize logical volumes, even while they are in use. This means that you can add or remove disks from a logical volume without having to repartition your disks.
- **Extensibility**: LVM can span multiple physical disks, which means that you can create logical volumes that are larger than the size of any single disk.
- **Fault tolerance**: LVM can be used to create **RAID arrays**, which provide data redundancy in case of a disk failure.

3.2.1 Implementing LVM, creating logical volumes (LVs), manipulating VGs& LVs

Logical volumes and **logical volume management** make it easier to manage disk space. If a file system that hosts a logical volume needs more space, it can be allocated to its logical volume from the free space in its volume group and the file system can be **resized**. If a disk **starts to fail**, a replacement disk can be registered as a **physical volume** with the **volume group** and the **logical volume's** extents can be migrated to the new disk.

Physical devices

Physical devices are the **storage devices** used to save data stored in a **logical volume**. These are block devices and could be disk partitions, whole disks, RAID arrays, or SAN disks. A device must be initialized as an LVM physical volume in order to be used with LVM. The entire device will be used as a physical volume.

Physical volumes (PVs)

You must **initialize** a device as a physical volume before using it in an LVM system. LVM tools segment physical volumes into physical extents (PEs),

which are **small chunks** of data that act as the smallest storage block on a physical volume.

Volume groups (VGs)

Volume groups are **storage pools** made up of **one or more** physical volumes. This is the functional equivalent of a whole disk in basic storage. A PV can only be allocated to a single VG. A VG can consist of **unused space** and any number of logical volumes.

Logical volumes (LVs)

Logical volumes are **created from free physical extents** in a volume group and provide the “storage” device used by applications, users, and the operating system. LVs are a collection of **logical extents** (LEs), which map to **physical extents**, the smallest storage chunk of a PV. By default, each LE maps to one PE. Setting specific LV options changes this mapping; for example, **mirroring** causes each LE to map to two PE’s.

The basic steps for adding a new logical volume are as follows.

1. If necessary, install a new hard drive.
2. Optional: Create a partition on the hard drive.
3. Create a physical volume (PV) of the complete hard drive or a partition on the hard drive.
4. Assign the new physical volume to an existing volume group (VG) or create a new volume group.
5. Create a new logical volume (LV) from the space in the volume group.
6. Create a filesystem on the new logical volume.
7. Add appropriate entries to `/etc/fstab` for mounting the filesystem.
8. Mount the filesystem.

3.2.2 Advanced LVM concepts (i.e. `system-config-lvm`)

LVM stands for *Logical Volume Management*. It is a **system of managing** logical volumes, or file systems, that is much more **advanced and flexible** than the traditional method of partitioning a disk into one or more segments

and formatting that partition with a file system.

One of the biggest advantages LVM has is that most operations can be done **on the fly**, while the system is running. Most operations that you can do with `gparted` require that the partitions you are trying to manipulate are not in use at the time, so you have to boot from the live cd to perform them. You also often run into the limits of the ms-dos partition table format with `gparted`, including only 4 primary partitions, and all logical partitions must be contained within one contiguous extended partition.

3.2.3 RAID concepts (Creating and managing a RAID-5 Array)

RAID (**Redundant Array of Independent Disks**) is a data storage technology that combines multiple disks to **provide data redundancy** and improve performance. RAID can be used to **protect data** from **disk failures**, and it can also be used to improve the performance of disk I/O.

There are several **different levels** of RAID, each of which offers different levels of data redundancy and performance. The most common levels of RAID are RAID 0, RAID 1, RAID 5, and RAID 6.

- RAID 0: RAID 0 **does not** provide any data redundancy. However, it can **improve** the performance of disk I/O by striping data across multiple disks.
- RAID 1: RAID 1 **mirrors** data across two disks. This means that if one disk fails, the data can still be read from the other disk.
- RAID 5: RAID 5 is a **striped** RAID array that also uses **parity** to protect data from disk failures. RAID 5 can **tolerate** the failure of one disk.
- RAID 6: RAID 6 is a **striped** RAID array that uses **double parity** to protect data from disk failures. RAID 6 can **tolerate** the failure of two disks.

Choosing between LVM and RAID

- ✓ LVM and RAID are both useful technologies for managing disk storage. The best choice for you will **depend on your specific needs**.
- ✓ If you need a flexible and extensible way to manage disk storage, then LVM is a good choice. LVM is also a good choice if you need to create RAID arrays.

- ✓ If you need to protect data from **disk failures**, then RAID is a good choice. RAID can also be used to improve the performance of disk I/O.

LVM is a good choice for flexible and extensible disk storage management for the following reasons:

1. **Dynamic Storage Management**: LVM allows for dynamic allocation and resizing of logical volumes, volume groups, and file systems. This means you can easily add or remove storage devices, expand or shrink volumes, and redistribute storage resources without interrupting ongoing operations. It provides a high degree of flexibility in managing storage capacity.
2. **Logical Volume Abstraction**: LVM abstracts the underlying physical storage devices and presents logical volumes to the operating system. This abstraction allows for easier management of storage resources, as you can treat logical volumes as independent entities rather than dealing with individual disks or partitions. It simplifies tasks such as resizing, moving, or creating snapshots of volumes.
3. **Pooling of Storage Resources**: LVM allows you to create volume groups that consist of multiple physical disks or partitions. This pooling of storage resources enables you to aggregate the capacity of multiple devices into a single logical volume. It provides increased flexibility in managing storage and allows for efficient allocation of space across multiple disks.

On the other hand, **RAID is a good choice** for protecting data from disk failures for the following reasons:

1. **Data Redundancy**: RAID provides redundancy by distributing data across multiple disks in different ways, such as mirroring or parity. This redundancy ensures that if one disk fails, the data can still be accessed from the remaining disks. RAID protects against data loss due to disk failures, enhancing data availability and reliability.
2. **Improved Fault Tolerance**: With RAID, the failure of a single disk does not result in immediate data loss. The redundant data on other disks can be used to reconstruct the missing information. Depending on the RAID level used, RAID can tolerate the failure of one or more disks, providing varying degrees of fault tolerance.
3. **Performance Enhancement**: Some RAID levels, such as RAID 0, offer performance benefits by striping data across multiple disks. This allows for parallel read and write operations, improving overall disk performance. While performance is not the primary focus of RAID for data protection, certain RAID configurations can deliver performance gains.

In general, LVM is a good choice **for flexible and extensible disk storage management**, while RAID is a good choice **for protecting data from disk failures**

and improving performance. The best choice for you will depend on your specific needs and requirements.